

# Agent 成本与性能优化：Token 预算、限流与成本归因

语言策略：code。正文三语言一致；成本追踪与滑动窗口限流代码按 Python 生态实现。地图节点：04 开发与工程化 / 成本与性能优化。配套文档：[大模型网关](#)、[Agent 可观测与追踪](#)。

## 0. 结论先说

- Agent 成本首先来自上下文膨胀，其次才是模型单价；先做上下文预算，再谈模型路由。
- 性能优化顺序：先测 P95 与 TTFT，再优化最慢的工具和最长上下文，最后才换更贵的模型。
- 任何优化都要绑定成本、延迟、成功率三个指标，防止“省了钱、坏了效果”。

## 1. 成本公式

```
1 单任务成本 = 输入 Token × 输入单价
2              + 输出 Token × 输出单价
3              + 工具调用成本
4              + 缓存命中节省
5              + 分摊基础设施成本
```

成本归因必须带 task\_id、model、prompt\_version、toolset\_version，否则无法定位是哪个版本变贵。

## 2. 成本优化杠杆

| 杠杆           | 做法                            | 典型收益  |
|--------------|-------------------------------|-------|
| 上下文预算        | 每轮计算预算，裁剪低优先级内容               | 高     |
| 模型分级         | 简单任务走小模型，复杂任务走强模型             | 高     |
| Prompt Cache | System Prompt 与工具 Schema 固定前缀 | 中高    |
| 工具返回压缩       | 只保留结构化摘要                      | 高     |
| 轮次上限         | 降低无效循环                        | 中     |
| 语义缓存         | 相似问题直接复用结果                    | 中，需防错 |
| 批处理          | 离线任务合并调用                      | 中     |

## 3. 性能优化顺序

- 建立基线：P50 / P95 / TTFT / TPOT / 单任务 Token；
- 找出最慢工具并设置超时与并行；
- 缩短上下文：裁剪历史、压缩工具返回；
- 流式输出降低首字延迟；
- 模型路由：强模型只处理复杂任务；
- 缓存与预取：Prompt Cache、检索预取。

## 4. 成本与性能门禁

- 单任务成本超预算 130% 告警；
- P95 超过 SLO 120% 告警；
- 成本优化上线前必须过评测门禁，成功率下降即回滚；
- 每月输出成本归因报告：按租户、任务类型、模型、Prompt 版本、工具。

## 5. Python 版成本追踪与限流

```
1 from collections import deque
2 from dataclasses import dataclass
3 from time import monotonic
4
5 @dataclass(frozen=True)
6 class TokenUsage:
7     input_tokens: int
8     output_tokens: int
9     input_price: float
10    output_price: float
11    tool_cost: float = 0.0
12
13    def cost(self) -> float:
14        return (self.input_tokens * self.input_price
15                + self.output_tokens * self.output_price
16                + self.tool_cost)
17
18 class SlidingWindowLimiter:
19     def __init__(self, max_requests: int, window_seconds: float) ->
20         None:
21         self.max_requests = max_requests
22         self.window_seconds = window_seconds
23         self.timestamps: deque[float] = deque()
24
25     def try_acquire(self, now: float | None = None) -> bool:
26         now = monotonic() if now is None else now
27         while self.timestamps and now - self.timestamps[0] >= self.
28             window_seconds:
29             self.timestamps.popleft()
30         if len(self.timestamps) >= self.max_requests:
31             return False
32         self.timestamps.append(now)
33         return True
```

## 6. 上线检查单

- 有单任务 Token 与成本预算；
- 有滑动窗口限流与并发上限；
- 有模型分级路由与降级策略；
- 有成本/延迟/成功率三个维度的回归对比；
- 缓存命中与错误率同时监控。

## 7. 延伸阅读

- [大模型网关](#)
- [Agent 可观测与追踪](#)
- [Agent 评测体系](#)